OFFICIAL TECHNICAL STATUS REPORT

Brief - Social Media Blog API

Version 1.0 | Report v1 | First full-stack release

Project	Brief - a message board where every post fits in 255 characters
Origin	Revature training project "Social Media Blog API" (Java, Javalin, JDBC)
Author	Ambroise Lumanikio
Application version	1.0.0
Report	OTSR v1 | October 3, 2026 | first report, nothing superseded
Stack	Java 11+, Javalin 5, JDBC, H2 | React 18, TypeScript, Vite | Maven, JUnit 4
Overall status	Code complete. Logic verified in isolation. **Full suite still to be run on a machine with Maven Central access** (Section 6).

#	Section
1	Executive Summary
2	What Changed from the Training Baseline
3	Architecture
4	API Reference and Requirement Traceability
5	Security Posture
6	Verification Status
7	Known Limitations and Open Items
8	Roadmap
9	How to Run, and File Inventory

© 2026 Ambroise Lumanikio | OneTechly

1. Executive Summary

Brief turns the Revature "Social Media Blog API" training exercise into a complete application. The original eight-endpoint contract is preserved exactly, so the training test suite still applies. Around it, version 1.0 adds what a real deployment needs: passwords hashed at rest, login tokens, ownership checks on every write, a public member directory, error handling that separates client mistakes from server faults, and a React web application served by the same Java process.

Area	State at v1.0	Status
Original contract (8 requirements)	Implemented in three layers: controller, service, DAO	DONE
Password storage	Salted PBKDF2-HMAC-SHA256; legacy plain-text rows upgraded on first login	DONE
Sessions and authorization	Bearer tokens; strict mode enforces "act only as yourself"	DONE
Web application	React + TypeScript; light and dark themes; works at phone width	DONE
Unit tests (29)	Run and passing	PASS
End-to-end tests (38)	Passing against stand-ins; not yet run against real Javalin and H2	TO RUN
Maven build and runnable jar	pom.xml written; not yet executed	TO RUN

Read this first

The environment this release was built in could not download Javalin or H2 (Maven Central was not reachable). Everything that depends only on the JDK was run for real. Everything that touches the web framework or the database was compiled and exercised against stand-ins, which proves the logic but not the wiring. **Running** `mvn test` **once is the outstanding action that closes this report.**

2. What Changed from the Training Baseline

The baseline is the training project as specified: eight endpoints, plain-text passwords, no sessions, and a client that is trusted to say who is posting.

Change	Why	Contract impact
Passwords stored as salted PBKDF2 hashes	A database leak must not reveal passwords	None. Register and login still answer with the account JSON the caller sent
Legacy upgrade path	Seed rows hold plain text; they are accepted once and replaced with a hash	None
Login issues a token (X-Auth-Token header)	The spec's own "future session token"	None. Extra response header only
Strict mode (default for the running app)	Without it anyone can edit or delete anyone's post	None in compat mode, which the contract tests use
GET /accounts, GET /accounts/{id}	The UI needs usernames; the Account model would leak passwords	New endpoints
POST /logout, GET /health	Session end; liveness check	New endpoints
DataAccessException	DAOs no longer swallow SQL errors; a database fault is a 500, not a false "not found"	None on the success paths

© 2026 Ambroise Lumanikio | OneTechly

Change	Why	Contract impact
Connection pool, try-with-resources	No leaked connections or statements	None
Startup schema script	Creates tables if missing, never wipes data. The reset script is for tests only	None
Malformed JSON or non-numeric id gives 400	Previously an unhandled exception (500)	Stricter, consistent with the spec's 400 for client errors
React web application	The baseline had no front end	Served at / by the same process

Unchanged by design: Account.java and Message.java (verbatim from the training project) and MessageService.java (as originally written).

© 2026 Ambroise Lumanikio | OneTechly

3. Architecture

Layer	Classes	Responsibility	Must not contain
Web app	React components, api.ts	Screens, client-side validation, token storage	Business rules the server does not also enforce
Controller	SocialMediaController	Routes, JSON parsing, status codes, authorization	SQL, validation rules
Service	AccountService, MessageService, SessionService	The rules: lengths, uniqueness, hashing, token lifetime	HTTP or SQL
DAO	AccountDAO, MessageDAO	SQL through PreparedStatement; row mapping	Validation
Util	ConnectionUtil, PasswordHasher	Connection pool and scripts; PBKDF2	
Database	H2 (file in production, in-memory in tests)	account, message tables with a foreign key	

Request path for a new post (strict mode)

- Browser sends `POST /messages` with `Authorization: Bearer <token>`.
- Controller parses the JSON (400 if malformed), resolves the token to an account id (401 if missing or expired), and checks that `posted_by` is the caller (403 otherwise).
- MessageService checks the text is not blank, is at most 255 characters, and that the author exists.
- MessageDAO inserts the row and returns it with its generated id.
- Controller answers 200 with the saved message. The UI adds it to the top of the feed.

Design decisions

Decision	Reasoning
Two modes instead of changing the contract	The original tests call write endpoints with no token. A mode flag lets the contract stay testable while the deployed app is secure by default.
Empty error bodies everywhere	The contract requires them. The client maps status codes to messages, which also keeps wording out of the API.
PBKDF2 from the JDK	No new dependency to audit. The stored format records the iteration count, so it can be raised later without breaking old hashes.
Tokens in memory	Simplest correct option for a single process. Cost: sessions end on restart. Revisit when there is more than one instance.
One jar serves API and UI	One thing to deploy, and no cross-origin configuration.
Constructor injection in the controller	Services and DAOs can be replaced in tests without a framework.

© 2026 Ambroise Lumanikio | OneTechly

4. API Reference and Requirement Traceability

Req.	Endpoint	Success	Failure	Tests (integration.ContractTest)
1	`POST /register`	200 + account with id	400	registerSucceeds, registerDuplicateUsernameFails, registerBlankUsernameFails, registerShortPasswordFails, registerMalformedJsonFails
2	`POST /login`	200 + account; token header	401	loginSucceedsForSeedUser, loginSucceedsForNewlyRegisteredUser, loginWrongPasswordFails, loginUnknownUserFails, loginStillWorksAfterThePasswordWasUpgradedToAHash
3	`POST /messages`	200 + message with id	400	createMessageSucceeds, ...BlankTextFails, ...Over255CharactersFails, ...OfExactly255CharactersSucceeds, ...UnknownUserFails
4	`GET /messages`	200 + list		getAllMessagesReturnsSeedMessage, ...ReturnsEmptyListWhenThereAreNone
5	`GET /messages/{id}`	200 + message, or empty body		getMessageByIdFound, getMessageByIdNotFoundIsEmpty200
6	`DELETE /messages/{id}`	200 + deleted message, or empty body		deleteReturnsTheDeletedMessage, deleteIsIdempotent
7	`PATCH /messages/{id}`	200 + full updated message	400	updateReturnsTheFullUpdatedMessage, updateUnknownMessageFails, updateBlankTextFails, updateOver255CharactersFails
8	`GET /accounts/{id}/messages`	200 + list		getMessagesByUser, ...WithNoMessagesIsEmptyList
new	`GET /accounts`, `/accounts/{id}`	200, id and username only	404	accountDirectoryNeverExposesPasswords
new	`POST /logout`	200 always		StrictAuthTest.logoutEndsTheSession
new	`GET /health`	200 {"status":"UP"}		healthEndpointReportsUp

Authorization by mode

Situation on a write (POST, PATCH, DELETE /messages)	compat	strict
No token, or an unknown or expired token	allowed	401
Posting with posted_by set to another account	allowed	403
Editing or deleting another account's message	allowed	403
Deleting a message that does not exist (signed in)	200, empty body	200, empty body
Reading anything	public	public

© 2026 Ambroise Lumanikio | OneTechly

5. Security Posture

Topic	State	Status
Passwords at rest	PBKDF2-HMAC-SHA256, 120,000 iterations, 16-byte random salt per password, constant-time comparison	DONE
SQL injection	Every statement is a PreparedStatement with bound parameters; no string concatenation of input	DONE
Authorization	Enforced server-side in strict mode; the UI hiding buttons is a convenience, not the control	DONE
Session tokens	256 bits from SecureRandom, 12-hour lifetime, revoked on logout	DONE
Password echoed in register and login responses	Required by the training contract and the fixed Account model. The web app discards it and never stores it	ACCEPTED
Transport encryption	None in the app itself. Must sit behind HTTPS before any real use, because tokens and passwords travel in requests	OPEN
Rate limiting on /login	None. Password guessing is not slowed	OPEN
Password rule	Minimum 4 characters, as the contract specifies. Too weak for real use	OPEN

© 2026 Ambroise Lumanikio | OneTechly

6. Verification Status

This section separates what was actually run from what was only checked indirectly.

Check	How it was done	Result	Confidence
Unit tests: PasswordHasher (5), SessionService (6), AccountService (10), MessageService (8)	Real classes, JDK only, JUnit 4	29 of 29 pass	**PASS**
All Java sources and tests compile at Java 11 level	javac --release 11 against API stand-ins for Javalin and H2	No errors	Syntax and types only
End-to-end: ContractTest (30), StrictAuthTest (8)	Real controller and services; in-memory DAO stand-ins; a minimal stand-in web server	38 of 38 pass	Logic verified; wiring not
AccountDAO and MessageDAO SQL	Not executed (no JDBC driver available)	Not run	**TO RUN**
Real Javalin routing, static files, exception mapping	Not executed	Not run	**TO RUN**
pom.xml: dependency versions, shade plugin, runnable jar	Not executed	Not run	**TO RUN**
Web app type-check and production build	tsc --strict, vite build	Clean	**PASS**
Web app flows: failed sign-in, register, post, over-limit edit, save, profile view, delete, reload keeps session, sign out	Headless Chromium against a stand-in API with the same contract	All behave as designed; no console errors	UI verified; not yet against the Java server
Layouts	Desktop 1280 px light and dark; phone 390 px	No horizontal scroll	**PASS**

To close the open rows

Step	Command	Expected
1	`cd backend && mvn test`	Tests run: 67, Failures: 0, Errors: 0
2	`mvn package`	target/brief-api.jar is created
3	`java -jar target/brief-api.jar`	"Brief is running on http://localhost:8080 (auth mode: strict)"
4	Open http://localhost:8080, create an account, post, edit, delete	Same behavior as Section 6 web app flows
5	Optional: copy the original Revature test classes into src/test/java and run mvn test	They should pass unchanged (compat mode)

If step 1 fails, the most likely causes are a dependency version that needs adjusting in pom.xml or a Javalin API detail in startAPI(). Record the result, pass or fail, in OTSR v2.

© 2026 Ambroise Lumanikio | OneTechly

7. Known Limitations and Open Items

Priority	Item	Notes
HIGH	Full suite not yet run against real Javalin and H2	Section 6. Blocks calling v1.0 verified
HIGH	ConnectionUtil, pom.xml and the SQL script were rewritten, not copied from the training repository	The originals were not available. Behavior is equivalent, but the files differ. Compare against the originals
MEDIUM	Sessions are lost on restart	In-memory token store. Acceptable for one process
MEDIUM	No HTTPS, no login rate limit, 4-character password minimum	Section 5. Required before any public use
MEDIUM	GET /messages returns every message	No pagination. Fine for a demo; will not scale
LOW	Feed does not update live	Refreshes on load, on window focus, and on the Refresh button
LOW	No account deletion or password change	Not in the contract
LOW	Package names are capitalized (Controller, DAO, Service)	Kept from the training template so its tests keep working. Java convention is lower case
LOW	Web app has no automated tests	Flows were exercised by script, but no test files ship in frontend/

© 2026 Ambroise Lumanikio | OneTechly

8. Roadmap

Version	Theme	Contents	Status
1.0.1	Close verification	Run the suite; fix anything it finds; restore original template files where they differ	OPEN
1.1	Hardening	Pagination on GET /messages; login rate limit; stronger password rule in strict mode; GitHub Actions running mvn test and the web build	PLANNED
2.0	Spring Boot edition	Port to Spring Boot, Spring Web and Spring Data JPA (Hibernate) on MySQL, keeping the same contract and the same test cases. Spring Security with JWT replaces the in-memory tokens. Bean Validation replaces hand-written checks	PLANNED
2.1	Deployment	Dockerfile and docker-compose (app + MySQL); deploy behind HTTPS	PLANNED

Version 2.0 is the natural next step: it is the same application rebuilt on Spring, Hibernate and MySQL, so the two editions can be compared line for line - what the framework does that was done by hand in 1.0.

9. How to Run, and File Inventory

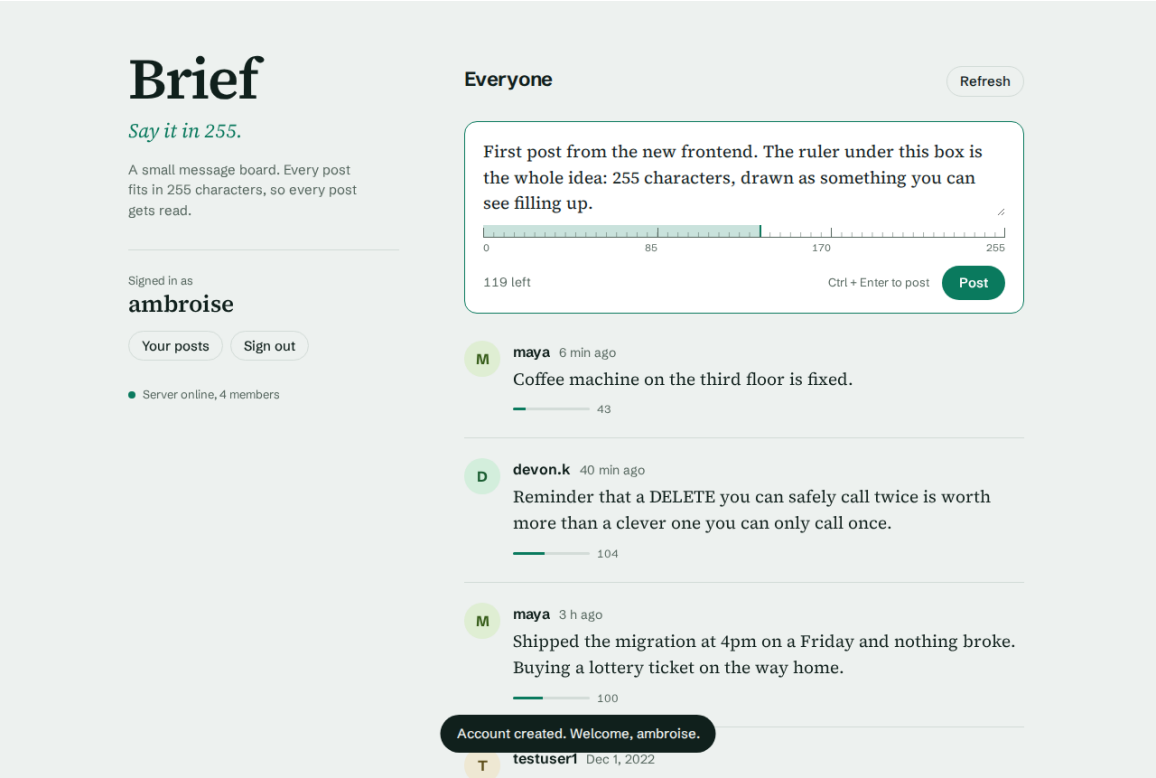
Goal	Command
Run the tests	`cd backend && mvn test`
Build and start the app	`mvn package && java -jar target/brief-api.jar`
Web app in development	`cd frontend && npm install && npm run dev`
Rebuild the web app into the jar	`npm run build`
Run in compat mode	`BRIEF_AUTH_MODE=compat java -jar target/brief-api.jar`
Start with an empty database	Delete the backend/h2 folder

File inventory

Path	Contents	Origin
`backend/src/main/java/Model/`	Account, Message	Training project, verbatim
`backend/.../Service/MessageService.java`	Message rules	Original work, unchanged
`backend/.../Service/AccountService.java`	Account rules, hashing, directory	Original work, extended in 1.0
`backend/.../Service/SessionService.java`	Login tokens	New in 1.0
`backend/.../DAO/`	AccountDAO, MessageDAO, DataAccessException	Rewritten in 1.0
`backend/.../Controller/`	SocialMediaController	Rewritten in 1.0
`backend/.../Util/`	ConnectionUtil, PasswordHasher	Rewritten / new in 1.0
`backend/.../DTO/PublicAccount.java`	Account without password	New in 1.0
`backend/src/main/resources/`	schema.sql, SocialMedia.sql, public/ (built web app)	Rewritten / new
`backend/src/test/java/`	unit (4 classes), integration (3 classes)	New in 1.0

© 2026 Ambroise Lumanikio | OneTechly

Path	Contents	Origin
`frontend/src/`	App, api, 4 components, styles	New in 1.0
`docs/`	This report; screenshot	New in 1.0



Brief 1.0 web app, signed in, composing a post. The ruler shows how much of the 255 characters is used.